

Interfaces declarativas con Jetpack Compose

Nombre

Joel Andrés Mendoza Buriticá

Instructor

Wilson Castro Gil

Ficha

3169892

Servicio Nacional de Aprendizaje SENA

Centro De Tecnología De La Manufactura Avanzada CTMA

Análisis y Desarrollo de Software

Medellín, 2026

Tabla de contenido

Actividad de activación · Del dato a la decisión.....	3
Seleccionen tres objetos ActividadFormativa con estados diferentes.	3
Para cada propiedad decidan si debe mostrarse, resumirse, transformarse o mantenerse oculta.	4
Identifiquen jerarquía visual: dato principal, secundario, estado y acción disponible.	4
Detecten un riesgo de accesibilidad y un riesgo de sobrecarga cognitiva.	5
Riesgo de accesibilidad:	5
Riesgo de sobrecarga cognitiva:	5
Elaboren el mapa dato → componente Compose → decisión que habilita.....	5
5.2 Actividad de contextualización · Pensar declarativamente	6
• Predice qué texto aparece para 60 y 100.....	6
• Explica qué entrada cambió y qué parte depende de ella.....	6
• Señala por qué escribir en una base de datos dentro del composable sería un efecto lateral peligroso.	6
• Formula una versión parametrizada que no dependa de variables globales.	6
5.3 Microprácticas de apropiación.....	7
5.4 Laboratorio integrador · Pantalla de actividades	10

Actividad de activación · Del dato a la decisión

Seleccionen tres objetos ActividadFormativa con estados diferentes.

```
ActividadFormativa(  
    id = 1,  
    titulo = "Clase de Wilson 1 porque solo Wilson da clases",  
    descripcion = "Variables y funciones",  
    progreso = 40,  
    diasRestantes = 5,  
    prioridad = Prioridad.MEDIA  
)
```

```
ActividadFormativa(  
    id = 2,  
    titulo = "Clase de Wilson 2",  
    descripcion = "Android Studio",  
    progreso = 3,  
    diasRestantes = 0,  
    prioridad = Prioridad.BAJA  
)
```

```
ActividadFormativa(  
    id = 3,  
    titulo = "Programación Orientada a Objetos",  
    descripcion = "Clases y objetos",  
    progreso = 100,
```

diasRestantes = -1,

prioridad = Prioridad.*ALTA*

)

Para cada propiedad decidan si debe mostrarse, resumirse, transformarse o mantenerse oculta.

Id = Ocultarse, no veo necesario que el usuario pueda ver este numero de identificación en una aplicación de notas.

Titulo = Mostrar, es indispensable que el usuario lo vea para saber que tarea es la que tiene pendiente.

Descripción = Resumir, es necesario mostrarlo, sí, pero no debería ser un texto demasiado largo.

Progreso = Transformar, lo ideal seria que fuera algo mas visual que un simple número.

diasRestantes = Mostrar, es completamente necesario que el usuario pueda ver cuantos días le quedan para entregar sus trabajos.

Prioridad = Mostrar, al igual que los días, es necesario que el usuario sepa la clasificación de sus trabajos pendientes para saber cuales hacer primero.

Identifiquen jerarquía visual: dato principal, secundario, estado y acción disponible.

Dato principal: titulo

Datos secundarios: descripcion y diasRestantes

Estado: progreso y prioridad

Acción disponible: abrir o seleccionar la tarjeta para consultar detalles de la actividad

Detecten un riesgo de accesibilidad y un riesgo de sobrecarga cognitiva.

Riesgo de accesibilidad: El contenido extenso sobre Scrum se presenta en un bloque de texto muy grande y con poca diferenciación visual entre títulos, subtítulos y contenido. Esto puede dificultar la lectura y comprensión de la información.

Riesgo de sobrecarga cognitiva: La pantalla presenta demasiada información al mismo tiempo. Además del resumen de actividades, se muestra un bloque extenso sobre Scrum con sus valores y 12 principios ágiles. Esto puede dificultar que el usuario identifique rápidamente la información relevante y la acción que debe realizar sobre sus actividades.

Elaboren el mapa dato → componente Compose → decisión que habilita.

Dato	Componente Compose	Decisión que habilita
titulo	Text	Identificar rápidamente cuál actividad necesita atención.
descripcion	Text (resumen)	Comprender el contenido de la actividad y decidir si abrirla para ver más detalles.
progreso	LinearProgressIndicator + Text (%)	Determinar cuánto trabajo falta para completarla.
diasRestantes	Text o AssistChip	Decidir si es necesario trabajar en la actividad de inmediato debido al tiempo disponible.
prioridad (BAJA, MEDIA, ALTA)	Badge, AssistChip o Text destacado	Establecer el orden de atención entre varias actividades.
id	No visible para el usuario	Identificar de forma única la actividad y usarla como clave estable en LazyColumn (key = { it.id }).

5.2 Actividad de contextualización · Pensar declarativamente

- *Predice qué texto aparece para 60 y 100.*

Si proceso es 60 entonces aparecerá “en proceso”, si es 100 aparecerá “completada”. Ya que este algoritmo indica que si una variable progreso es mayor o igual a 100 esta se marcará como completada.

- *Explica qué entrada cambió y qué parte depende de ella.*

La entrada que cambio fue la variable progreso, esta es la que permite calcular esta condición para saber si la actividad está completada o no. Toda la función en si depende de ella por lo que mencione anteriormente.

- *Señala por qué escribir en una base de datos dentro del composable sería un efecto lateral peligroso.*

Porque los composables pueden ser repetidos, esto significa que podríamos repetir la escritura de la base de datos una y otra vez generando datos duplicados, consumo innecesario de recursos y comportamientos inesperados en la aplicación.

- *Formula una versión parametrizada que no dependa de variables globales.*

La versión que tenemos dentro de la guía ya esta parametrizada porque no hay variables globales fuera de este así que no es necesario hacerlo.

```
@Composable
```

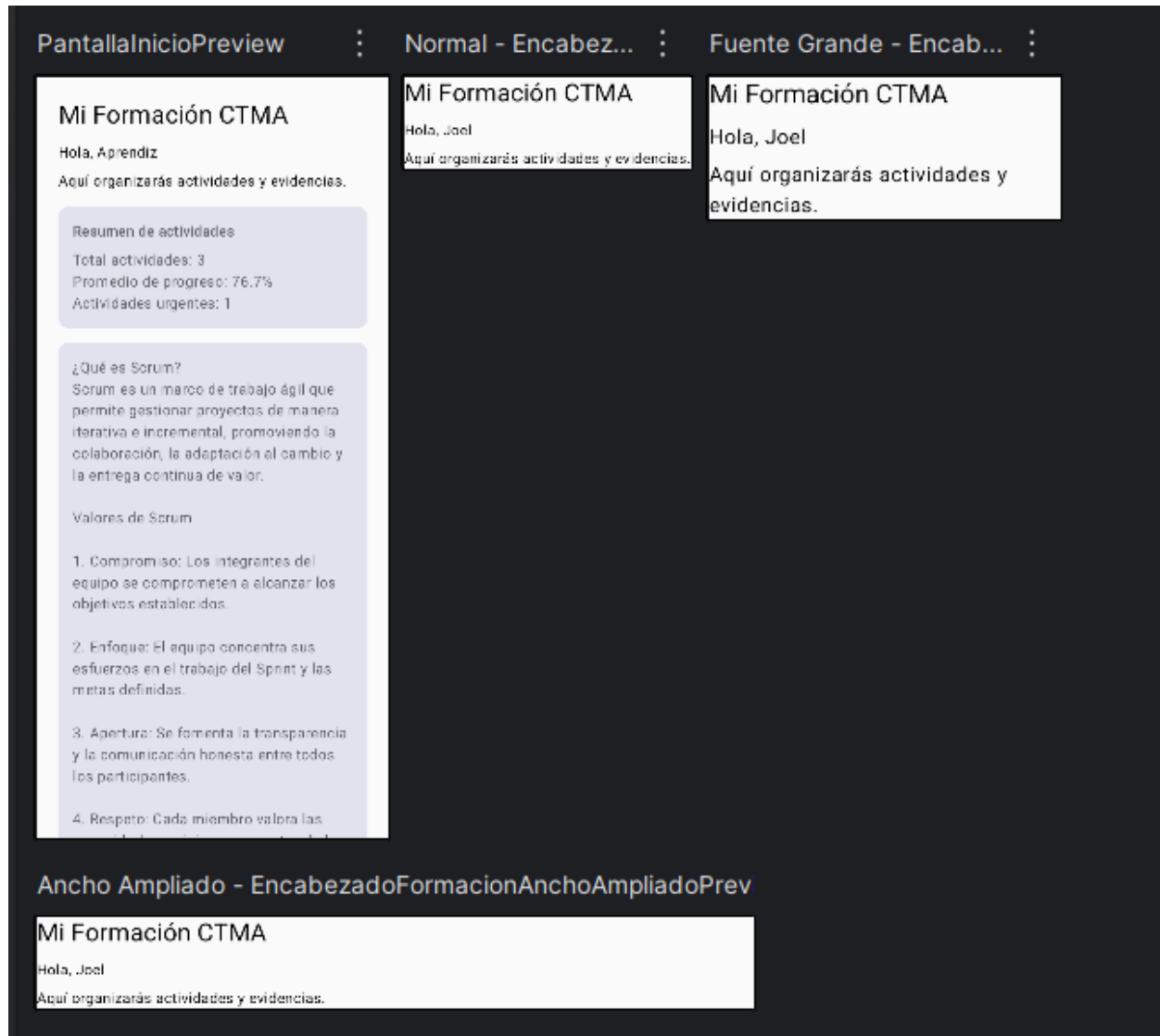
```
fun EstadoActividad(progreso: Int) {
```

```
    val texto = if (progreso >= 100) "Completada" else "En proceso"
```

```
    Text(text = texto)
```

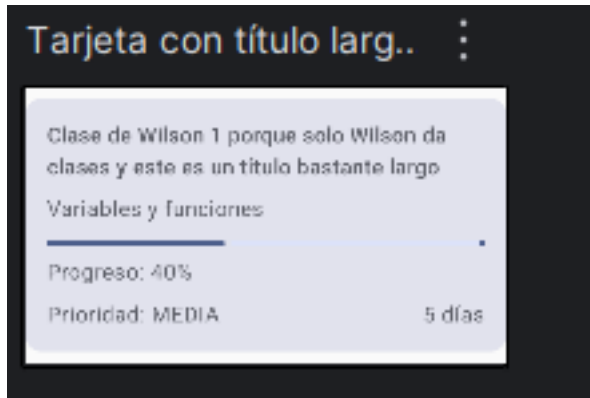
```
}
```

5.3 Microprácticas de apropiación



```
Card(
    modifier = Modifier
        .fillMaxWidth()
        .clickable { }
        .padding(top = 16.dp)
) {
```

Creando este ejemplo para explicarlo puedo hacerlo mas sencillo, por mi entendimiento puedo suponer que funciona en orden de cascada, por lo que si el padding estuviera primero el comportamiento del área sería diferente.



El proyecto ya cumple con la D ya que, por defecto el Android estudio crea un ColorScheme y Typography centralizados.

```
Image(
    painter = painterResource(id = R.drawable.ilustracion_formacion),
    contentDescription = "Ilustración relacionada con la formación y el estudio"
    modifier = Modifier
        .fillMaxWidth()
        .padding(vertical = 8.dp)
)
```


Mi Formación CTMA

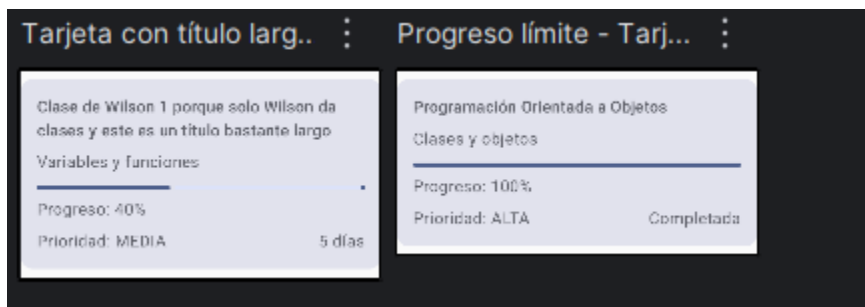
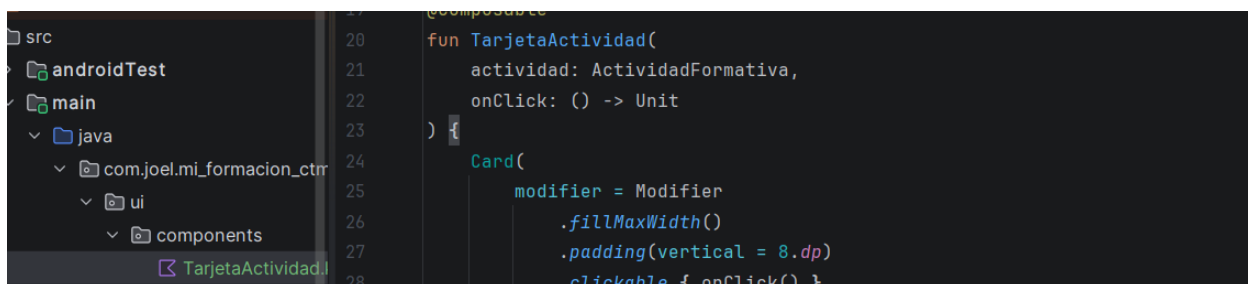
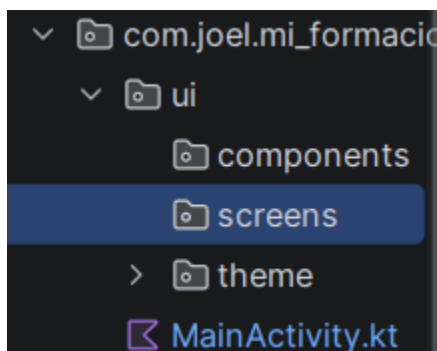
Hola, Aprendiz



Aquí organizarás actividades y evidencias.

Lista de actividades ...	
Clase de Wilson 1 porque solo Wilson da clases	
Variables y funciones	
Progreso: 40%	
Prioridad: MEDIA	5 días
Clase de Wilson 2	
Android Studio	
Progreso: 90%	
Prioridad: BAJA	3 días
Programación Orientada a Objetos	
Clases y objetos	
Progreso: 100%	
Prioridad: ALTA	Completada

5.4 Laboratorio integrador · Pantalla de actividades





```
18 @Composable
19 fun PantallaActividades(
20     actividades: List<ActividadFormativa>
21 ) {
22     Scaffold(
23         modifier = Modifier.fillMaxSize()
24     ) { paddingValues ->
25
26         LazyColumn(
27             modifier = Modifier
28                 .fillMaxSize()
29                 .padding(paddingValues)
30                 .padding(horizontal = 16.dp)
31         ) {
32             item {
33                 Text(
34                     text = "Mis actividades"
35                 )
36             }
37         }
38     }
39 }
```

PantallaActividadesPreview

Mis actividades

Clase de Wilson

Variables y funciones

Progreso: 40%

Prioridad: MEDIA5 días

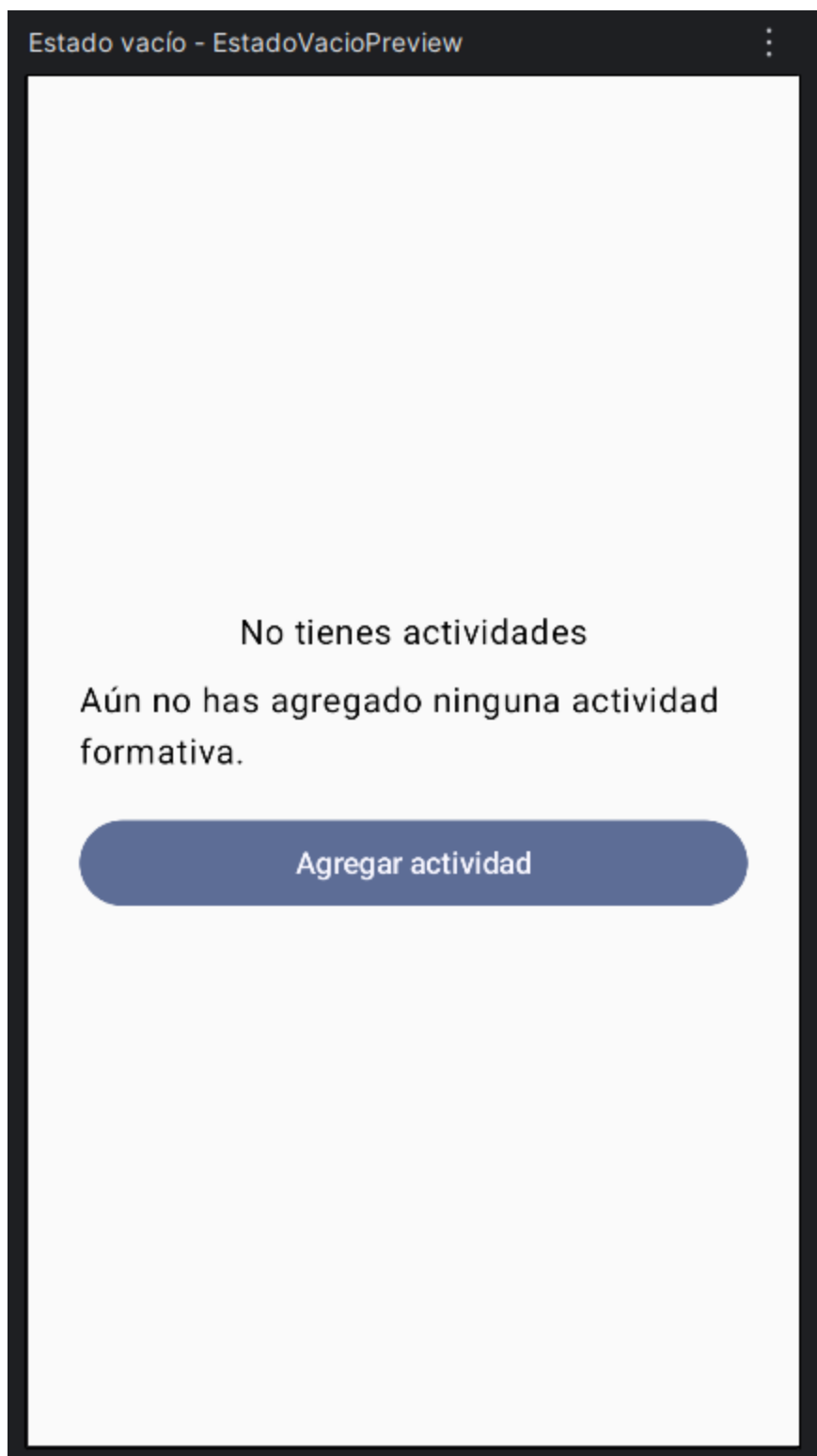
Programación Orientada a Objetos

Clases y objetos

Progreso: 100%

Prioridad: ALTACompletada

```
items(  
    items = actividades,  
    key = { it.id }  
) { actividad ->  
    TarjetaActividad(  
        actividad = actividad  
        onClick = { }  
    )  
}
```



```
@Composable
fun EstadoVacio(
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier.padding(all = 24.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(
            text = "No tienes actividades"
        )

        Text(
            text = "Aún no has agregado ninguna actividad formativa.",
            modifier = Modifier.padding(top = 8.dp)
        )

        Button(
            onClick = { },
            modifier = Modifier
                .fillMaxWidth()
                .padding(top = 16.dp)
        ) {
            Text(
                text = "Agregar actividad"
            )
        }
    }
}
```

```

LazyColumn(
    modifier = Modifier
        .fillMaxSize()
        .padding(paddingValues)
        .padding(horizontal = 16.dp)
) {
    item {
        Text(
            text = "Mis actividades",
            style = MaterialTheme.typography.headlineSmall,
            modifier = Modifier.padding(vertical = 16.dp)
        )
    }
}

```

```

fun TarjetaActividad(
    actividad: ActividadFormativa,
    onClick: () -> Unit
) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(vertical = 8.dp)
            .semantics {
                contentDescription =
                    "Actividad ${actividad.nombre}"
            }
        .clickable { onClick() }
    )
}

```


Mis actividades

Clase de Wilson

Variables y funciones



Progreso: 40%

Prioridad: MEDIA

5 días

Programación Orientada a Objetos

Clases y objetos



Progreso: 100%

Prioridad: ALTA

Completada

Diseño de interfaces

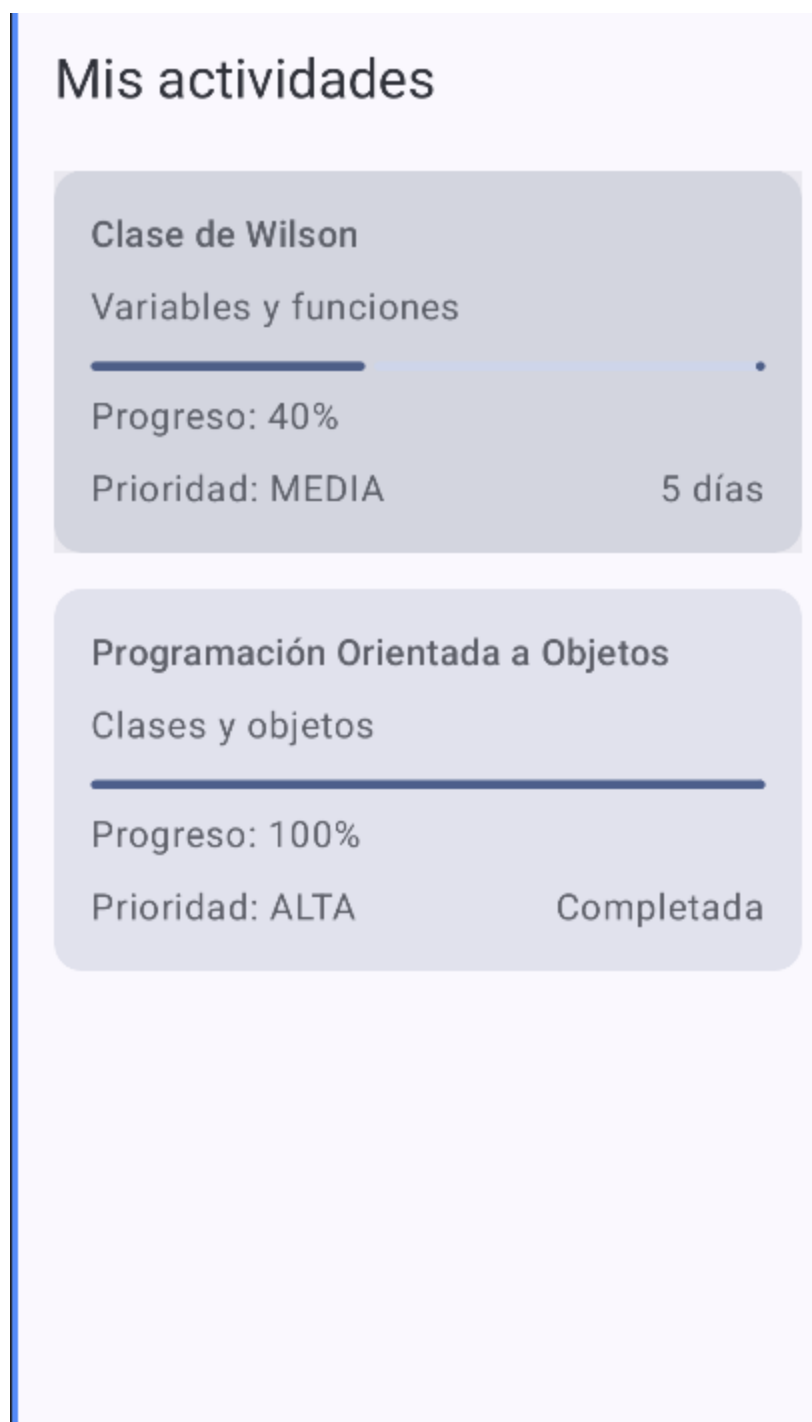
Componentes visuales en Compose



Progreso: 60%

Prioridad: MEDIA

7 días



Se probó la aplicación en un teléfono y en una pantalla de mayor ancho. En ambos casos las tarjetas, textos y títulos se mostraron correctamente sin recortes. También se comprobó el escalado de fuente. Como hallazgo, se observó que en pantallas anchas existe espacio que podría aprovecharse mejor, aspecto que se mejorará en el reto adaptable.